MRSpinDynamics

NMR and NQR simulations in inhomogeneous fields

MATLAB reference and Python port

MATLABSpinDynamics

User Manual

Reference MATLAB implementation for NMR spin-dynamics simulations

Classical isochromat simulations for uncoupled spin-1/2 nuclei in non-uniform and time-varying magnetic fields, with ideal, tuned, untuned, and impedance-matched probe workflows.

June 2026

Contents

1 Physical Scope and Assumptions

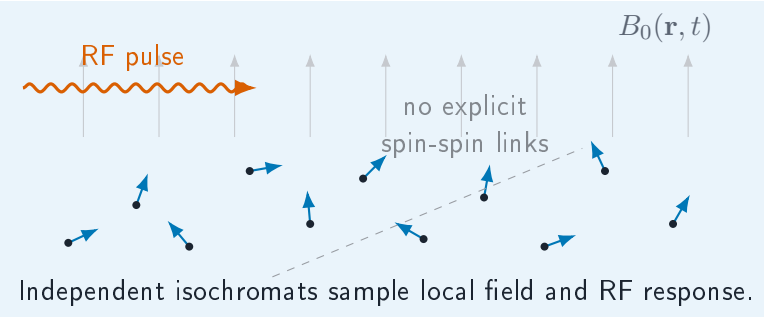
MATLABSpinDynamics models a bath of uncoupled spin-1/2 nuclei in a main field $B_0(\mathbf{r}, t)$ that may be non-uniform, time-varying, or both. The sample is discretized into isochromats. Each isochromat represents a small subensemble that shares an offset, RF sensitivity, relaxation constants, and probe response.

Modeled directly. Independent spin-1/2 magnetization vectors; offset distributions from $B_0(\mathbf{r}, t)$; RF rotations; free precession; phenomenological relaxation; diffusion in selected workflows; phase encoding; and probe-circuit transmit/receive filtering.

Not modeled explicitly. Spin-spin coupling, J-coupling networks, dipolar coupling, spins with $I > 1/2$, quadrupolar transitions, multi-quantum coherences, density-matrix entanglement, chemical exchange, and microscopic spin noise. These effects only enter indirectly if represented by effective relaxation constants, diffusion terms, or supplied field maps.

This is a Bloch/isochromat simulator, not a general quantum spin Hamiltonian solver. It is most useful when the experiment can be described by the motion of classical magnetization vectors under prescribed fields and circuit response.

1.1 Spin Bath Cartoon



1.2 Practical Consequences

- Use offset grids or field maps to represent B_0 inhomogeneity.
- Use transmit and receive B_1 maps or probe models to represent RF non-uniformity and circuit filtering.
- Use T_1, T_2, diffusion, or effective field maps to capture physics outside the explicit uncoupled-spin model.
- Avoid using this code for phenomena that require explicit coupled-spin quantum pathways.

2 Installation Requirements

2.1 MATLAB Path Setup

No packaging step is required. From the `MATLABSpinDynamics` directory, add the active Version 3 code tree to the MATLAB path:

```
repo = pwd;
addpath(genpath(fullfile(repo, 'Version_3', 'code')));
```

For a script launched elsewhere, replace `pwd` with the absolute path to `MATLABSpinDynamics`. The exact minimum MATLAB release has not been pinned. Use a recent MATLAB release for the current Version 3 scripts and graphics.

2.2 Core and Optional Dependencies

Requirement	When it is needed
Base MATLAB	Core ideal spin dynamics, many low-level rotation/echo helpers, and many basic probe examples.
Parallel Computing Toolbox	Workflows that use `parfor`, including many Q sweeps, mistuning sweeps, imaging scripts, CPMG-IR scripts, and optimization repeats. Convert `parfor` to `for` for smaller serial runs.
Optimization Toolbox	Matched-probe network design and pulse optimization workflows using `fmincon` or `optimoptions`.
Image Processing Toolbox	Imaging examples that call `imresize` or `rgb2gray`.
MATLAB Coder and C/C++ compiler	Optional MEX/code-generation build scripts under `Version_3/code/mex`. Configure the compiler with `mex -setup`. Generated MEX/codegen outputs are local build artifacts and are ignored by Git.
`export_fig`	Optional third-party File Exchange helper used by some plotting/export scripts through `safe_export_fig`. Missing `export_fig` causes exports to be skipped with a warning.

Use `ver` in MATLAB to inspect installed toolboxes before running a workflow. Missing optional toolboxes do not prevent the base simulation functions from being used.

3 Repository Map

Path	Purpose
`Version_3/code`	Recommended active MATLAB code.
`Version_2`	Earlier updated implementation kept for historical comparison.
`Version_1`	Legacy original implementation.
`docs/QUICK_START.md`	Practical starting guide for common MATLAB runs.
`docs/VERSION_GUIDE.md`	Map of active, legacy, and repeated functions.
`docs/VERSION_3_WORKFLOWS.md`	Workflow-oriented guide to Version 3 scripts.
`docs/SPEED_AUDIT.md`	Performance observations and acceleration notes.

4 Model Conventions

4.1 Isochromat Evolution

The simulation discretizes the sample into offsets $\Delta\omega_k$ and, when needed, transmit/receive sensitivities. For a rectangular RF interval with amplitude ω_1, phase ϕ, and duration t, the rotating-frame effective field is

$$\mathbf{\Omega}_k = \begin{bmatrix} \omega_1 \cos\phi \\ \omega_1 \sin\phi \\ \Delta\omega_k \end{bmatrix}, \qquad \Omega_k = \|\mathbf{\Omega}_k\|.$$

The magnetization update is a rotation by $\Omega_k t$ about $\mathbf{\Omega}_k/\Omega_k$. The axis and angle generally differ across isochromats because offsets and RF sensitivities differ.

4.2 Normalized Time

The bare spin-dynamics functions use normalized units with $\omega_1 = 1$. Thus a nominal 90-degree pulse has duration

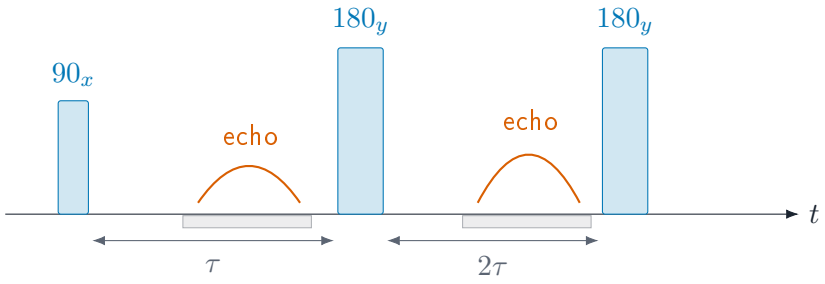
$$t_{90} = \frac{\pi}{2},$$

and a nominal 180-degree pulse has duration

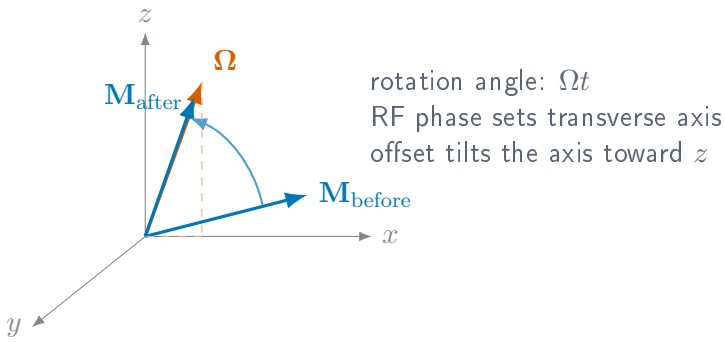
$$t_{180} = \pi.$$

Probe-aware workflows often use physical seconds internally or in their parameter structures. When mixing bare and probe-aware code, check whether each function expects normalized or physical time.

4.3 Pulse Timing Cartoon



4.4 Rotation Cartoon



4.5 Relaxation and Diffusion

Default bare functions omit relaxation and diffusion for speed. Relaxation variants apply exponential updates such as

$$M_{xy}(t+\Delta t) = M_{xy}(t)\exp\left(-\frac{\Delta t}{T_2}\right)\exp(-\mathrm{i}\Delta\omega\,\Delta t),$$

$$M_z(t+\Delta t) = M_{\mathrm{eq}} + (M_z(t) - M_{\mathrm{eq}})\exp\left(-\frac{\Delta t}{T_1}\right).$$

Diffusion-aware workflows add attenuation or state updates specific to the selected pulse sequence and probe model.

5 First Workflow

5.1 A Minimal Ideal CPMG Calculation

The lower-level MATLAB interface builds pulse intervals and passes them to rotation/echo helpers. A stripped-down CPMG calculation looks like this:

```
numpts = 2001;
maxoffs = 20;
del_w = linspace(-maxoffs, maxoffs, numpts);

tacq = 5*pi;
window = sinc(del_w*tacq/(2*pi));
window = window ./ sum(window);

texc = [pi/2 -1];
pexc = [pi/2 0];
aexc = [1 0];

tref = pi*[3 1 3];
pref = [0 0 0];
aref = [0 1 0];

neff = calc_rot_axis_arba3(tref, pref, aref, del_w, 0);
masy1 = sim_spin_dynamics_asymp_mag3(texc, pexc, aexc, neff, del_w, tacq);
masy2 = sim_spin_dynamics_asymp_mag3(texc, pexc + pi, aexc, neff, del_w, tacq);
masy = (masy1 - masy2)/2;

snr = trapz(del_w, abs(masy).^2);
[echo, tvect] = calc_time_domain_echo(masy, del_w);
```

Many production scripts use `set_params_*` helpers instead of spelling out every interval by hand. Those helpers create MATLAB structures, usually named `sp`, `pp`, and sometimes `params`, that carry simulation, pulse, and circuit settings.

5.2 Probe-Aware Workflows

Probe-aware scripts model untuned, tuned, and impedance-matched probes. The main caution is unit conversion: bare spin-dynamics routines commonly expect normalized time, while circuit workflows may use physical seconds and convert internally. Always check the function help and nearby workflow script before mixing pieces from different folders.

6 Workflow Map

Folder or family	Typical use
`calc_rot`	Effective rotation axes, rotation matrices, critical offsets, and probe-aware pulse response helpers.
`calc_echo`	Frequency-domain to time-domain echo conversion.
`sim_spin_dynamics_asymp`	Asymptotic and excitation spin-dynamics kernels for ideal CPMG-style studies.
`cpmg_*` and probe folders	Ideal, tuned, untuned, and matched CPMG sequence calculations.
`CompareQ`	Quality-factor sweeps for tuned and matched probes.
`CompareMistuned`	Probe mistuning studies.
`CPMG_IR`	CPMG inversion-recovery workflows.
`Images` and imaging scripts	Phase-encoded imaging examples and field-map helpers.
`Sim_Diffusion` and `calc_macq_diff`	Diffusion-aware matched probe spin-echo and CPMG variants.
`time_varying_field`	CPMG under prescribed time-varying fields.
`Wurst_Inversion`	WURST inversion and matched WURST-CPMG studies.
`opt_pulse`	Refocusing and excitation pulse optimization.
`mex`	Optional compiled-kernel/code-generation experiments.

7 Function Family Reference

7.1 Bare Spin Dynamics

Start here when developing or validating a new pulse sequence:

```
calc_rotation_matrix
calc_rot_axis_arba3
calc_rot_axis_arba4
sim_spin_dynamics_asymp_mag3
sim_spin_dynamics_exc
calc_time_domain_echo
calc_time_domain_echo_arb
```

These functions expose the core interval representation: durations, RF phases, RF amplitudes, offsets, and acquisition flags. They are fast and close to the underlying Bloch/isochromat model, but they leave more setup work to the user.

7.2 Parameter Constructors

Many workflow scripts use `set_params_*` functions to assemble default simulation and pulse structures:

```
set_params_ideal
set_params_tuned_orig
set_params_untuned_orig
set_params_matched_orig
set_params_tuned_spa
set_params_untuned_spa
set_params_matched_spa
set_params_matched_SS
```

Treat these functions as executable documentation. They are often the clearest place to find default pulse lengths, acquisition windows, Q values, matching network values, and relaxation settings.

7.3 Probe Models

```
tuned_probe_lp
tuned_probe_rx
tuned_probe_rx_tf
untuned_probe_lp
untuned_probe_rx
untuned_probe_rx_tf
matching_network_design2
find_coil_current
find_coil_current_WURST
matched_probe_rx
```

The tuned and untuned helpers model transmit/receive filtering directly. The matched-probe path includes circuit design and coil-current calculations, and therefore may require Optimization Toolbox for design-time routines.

7.4 Pulse Shapes and Optimization

```
create_WURST
quantize_phase
untuned_pulse_adjust
opt_ref_pulse_tuned
opt_ref_pulse_untuned
opt_ref_pulse_matched
opt_exc_pulse_tuned
```

Optimization scripts often use `parfor` for repeated random starts and `fmincon` for local constrained optimization. For quick experiments, reduce the number of starts or replace `parfor` loops with ordinary `for` loops.

7.5 Generated or Compiled Kernels

The `mex` folder contains optional build scripts for compiled kernels. These are not required for ordinary use. Generated MEX files and codegen reports are ignored release artifacts. Rebuild them locally with MATLAB Coder and a configured compiler:

```
mex -setup
```

8 Validation and Porting Notes

The sibling Python package uses this MATLAB Version 3 tree as its numerical reference. When changing MATLAB behavior, update the relevant fixture scripts and document whether the Python port should follow the new behavior or preserve older compatibility.

Good validation practice is:

- create a small MATLAB script that exercises the exact function family;
- save compact arrays rather than large figures or workspace dumps;
- test ideal kernels before adding probe response, relaxation, diffusion, or optimization;
- record unit conventions, especially normalized time versus seconds.

9 Known Caveats

This repository contains multiple generations of related functions. Names are sometimes repeated across folders. Prefer the active Version 3 code tree for new work, check `docs/VERSION_GUIDE.md` when a function appears in more than one place, and keep local path edits out of numerical changes.

The code is strongest as a research reference and simulation toolkit. It is not currently packaged as a MATLAB Toolbox app, and it does not try to hide every experimental script behind a single stable API.